PatentPulse: A Java System for Research Version Tracking and File Integrity

Mohit Sharma, Meet, Jay, Nitesh and Mann Jain
Department of Artificial Intelligence and Data Science
Arya College of Engineering and IT, Jaipur, Rajasthan, India
Affiliated to Rajasthan Technical University, Kota

Design and implementation paper draft

Abstract- Research review requires a stable connection between a submitted file, the metadata describing it and the decisions made about that version. This paper presents PatentPulse, a Servlet and JSP application that maintains this connection using relational version records, controlled file storage and SHA-256 fingerprints. Researchers create drafts and submit versions. Administrators assign reviewers, and the service calculates an internal status from their decisions. Authorised users can compare a candidate file with a recorded fingerprint. The design uses Java 17, JDBC and MySQL, with H2 and embedded Tomcat for the supplied test environment. We examine the implementation, review its trust assumptions and describe an evaluation protocol. The archive reports 11 passing tests; a fresh execution and deployment study are still required. The contribution is an integrated educational prototype, with no claim of a new cryptographic method, measured performance advantage or legal proof of invention.

Index Terms- research management, document versions, SHA-256, Servlet, JSP, internal review, access control.

I. INTRODUCTION

A file name alone is a weak link between research content and reviewer feedback. A document may change while retaining the same visible name, and comments can become detached from the version they describe. A useful internal system should preserve submitted snapshots, identify content consistently and limit who can inspect each version. PatentPulse models the research item and its individual submissions as separate entities.

The design question is whether a small Java web application can support version-specific review and exact-file comparison without relying on a distributed ledger. Our scope is an institutional prototype. We do not evaluate legal patentability, plagiarism or semantic similarity. A MATCH result concerns the candidate bytes and stored digest under the system's trust assumptions.

II. BACKGROUND AND DESIGN METHOD

NIST defines the SHA family in the Secure Hash Standard [1]. PatentPulse uses Java's SHA-256 implementation to derive a fixed-length digest from an input stream. The digest is useful for detecting changed file content, but storing it beside a file does not protect against an administrator who can alter both records. The implementation therefore treats the server and database operator as trusted.

Provos and Mazières introduced bcrypt as a password-hashing construction with adaptable computational cost [2]. PatentPulse uses BCrypt for account credentials rather than using the document fingerprint as a password mechanism. This separation reflects different purposes: document comparison should be efficient, while password verification should impose a deliberate work factor.

OWASP's upload guidance describes several controls that belong together, including file restrictions, generated filenames, size limits and storage choices [3]. The prototype implements a subset and records remaining gaps. It does not include antivirus scanning or a sandbox for opening uploaded files. Merely accepting a filename extension is insufficient evidence that its content is harmless.

Patent administration also has a distinct boundary. IP India describes patentability conditions and statutory exclusions [4]. WIPO explains territorial rights and the PCT application route [5,6]. These sources inform the language used in the interface and receipts. They do not establish that PatentPulse submissions have any particular legal evidentiary status.

A. Method

This paper uses source inspection of the supplied project [7] and the cited technical literature. We trace request handlers to service operations, database constraints and file-storage actions. We distinguish implemented behaviour from proposed extensions and reported test results from fresh measurements. No interviews, user study or production traffic dataset was supplied.

B. Contribution

The prototype combines numbered metadata snapshots, per-version access rules, a two-reviewer decision policy and explicit file comparison in a small Java teaching project. These components are established techniques. The contribution is their integration and the analysis of where their guarantees end, rather than a novelty claim about hashing, peer review or patent systems.

III. SYSTEM MODEL AND ALGORITHMS

The relational model contains users, blueprints, document_versions, review_assignments, review_votes and audit_logs. Each version records the original name, generated storage name, size, digest, timestamp and metadata snapshot. The file bytes reside outside the web root. A researcher may create several submissions under one parent record without overwriting the snapshots associated with older submissions.

A. Submission and comparison

Submission first validates the multipart file and writes it with a generated filename. The application checks the content according to its permitted format class, then calculates the fingerprint. A database transaction allocates the version number and inserts metadata and an audit event. If persistence fails, the controller attempts to remove the newly written file. A process crash remains a separate recovery concern.

For verification, let H be SHA-256, F the candidate file and h the digest recorded for an authorised version. The application computes c = H(F). It returns MATCH when c equals h and MISMATCH otherwise. The comparison does not parse the research or judge its meaning. For n input bytes, hashing takes O(n) work and fixed-buffer auxiliary memory. The upload implementation makes several streaming passes for writing, validation and hashing, so storage I/O remains relevant even though asymptotic memory use is bounded.

B. Access and review rules

A version is accessible to its owner, an administrator, or a reviewer assigned to that version. Each reviewer assignment accepts one vote. If a is the number of approvals and r the number of rejections, the policy returns NEEDS_REVISION when r is positive, INTERNALLY_REVIEWED when a is at least two and otherwise IN_REVIEW. Final states reject further decisions. Draft is an inferred condition of a research item with no submitted version, not a value in the version-status column.

C. Trust boundary

HTTPS configuration, operating-system permissions and database protection depend on deployment. Application-level immutability does not stop privileged database edits. Receipts use server records and lack digital signatures or independent timestamp attestation. FileServlet performs authorisation and download streaming without automatic stored-file rehashing. Verification is a separate comparison-file operation.

IV. EVALUATION EVIDENCE AND LIMITATIONS

TABLE I. EVALUATION EVIDENCE AND REMAINING WORK

Question	Available evidence	Remaining work
Correct digest behaviour	4 supplied hash tests and the archived passing summary.	Rerun on the final branch and retain the test report.
Correct review policy	4 supplied review-policy tests.	Check concurrent requests on the deployment database.
Connected HTTP workflow	3 supplied HTTP workflow tests using H2 and Tomcat.	Repeat with configured MySQL and external Tomcat.
Performance and usability	No benchmark dataset or participant results supplied.	Measure latency, resource use and task completion.
Security of deployment	Controls visible in source; no independent audit supplied.	Review TLS, secrets, throttling, upload scanning and recovery.

The archive reports 11 tests passed on 6 October 2026, with no failures, errors or skipped tests [7]. This paper does not treat that statement as a new independent run. H2 compatibility checks are useful for functional development but do not establish equivalent locking, SQL behaviour or operational reliability on MySQL.

A. Proposed reproducible evaluation

Record the source commit and runtime versions. Run mvn clean verify and retain the generated reports. Repeat a defined workflow with original and altered files, unauthorised version identifiers, repeated votes and invalid uploads. For performance evaluation, use a published matrix of file sizes up to the configured limit, repeat each trial, and report the median and tail latency together with hardware and concurrency. No values should be filled until those trials have run.

B. Threats to validity

The current evidence covers a bounded test suite and a supplied release snapshot. It does not show the accuracy of the team's historical daily reports, performance under realistic load, adoption by researchers or resistance to privileged tampering. Lack of malware scanning and session-scoped throttling restrict claims about public deployment.

V. CONCLUSION

PatentPulse demonstrates how a version-oriented review workflow can be expressed in Servlet, JSP and JDBC code. The model keeps decisions attached to explicit submissions and supports controlled comparison of candidate files. Further evaluation should focus on the actual deployment environment and recovery behaviour before the prototype is used for sensitive institutional records.

REFERENCES

[1] NIST. Secure Hash Standard. FIPS PUB 180-4, 2015. https://doi.org/10.6028/NIST.FIPS.180-4

[2] N. Provos and D. Mazières. A Future-Adaptable Password Scheme. USENIX Annual Technical Conference, 1999. https://www.usenix.org/conference/1999-usenix-annual-technical-conference/future-adaptable-password-scheme

[3] OWASP Foundation. File Upload Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[4] IP India. Basics of Patents. https://ipindia.gov.in/pages/patents/learn/basics-of-patents

[5] WIPO. Frequently Asked Questions on Patents. https://www.wipo.int/en/web/patents/faq_patents

[6] WIPO. Patent Cooperation Treaty. https://www.wipo.int/en/web/treaties/registration/pct/index

[7] PatentPulse source archive supplied by the team. Java source, schema.sql, pom.xml and docs/TEST-REPORT.md. Original test report dated 6 October 2026.

Web references accessed 12 October 2026. This manuscript is an unpublished project paper draft. No journal acceptance, DOI assignment, peer review or experimental outcome beyond the attributed archive report is claimed.